APIConnect-catalog

How the GitOps pipeline for IBM API Connect works — a practical guide

TARGET PLATFORM
IBM API Connect 10.0.8.x — three instances (dev, test, prod)
DataPower API Gateway

CI/CD
Azure DevOps Pipelines, self-hosted Linux x64 agent (on-prem) or Microsoft-hosted

REPOSITORY
APIConnect-catalog (this bundle)

DOCUMENT VERSION
1.3.1 — 28 September 2026

AUDIENCE
Platform / integration engineers who operate the pipeline, and API developers who submit changes

COMPANION FILES
README.md (reference), config/topology.yml, .azure-pipelines/, scripts/, tools/

This document explains the design and the day-to-day use. The README in the repository is the concise technical reference; where they differ, the repository files are authoritative.

Contents

1. Why this exists — the problem and the three decisions
2. The big picture — three instances, four folders, five catalogs
3. API Connect concepts you need — API, Product, Plan, Catalog, Instance
4. Repository layout — where things live and why
5. Life of a change — from a dev edit to production
6. The four validation stages
7. Backward compatibility and versioning policy
8. Publishing — five independent stages, one folder each
9. Promotion by pull request — dev → integ → rc → prod
10. One file, five catalogs — catalog-specific properties
11. Security — ingress URLs in Git, secrets in Azure DevOps
12. Setting up an on-prem agent from the offline bundle
13. Setting up Azure DevOps
14. Day-2 operations — add, change, promote, break, retire, roll back
15. Troubleshooting — reading a failed run
16. Before the first publish — items to verify on each instance
17. Appendix — scripts, tools and versions

> **Reading guide**
> Developers who only need to submit API changes can read sections 2, 5, 6, 7, 9 and 14. Operators should read everything once; sections 12, 13 and 16 are the set-up checklist.

1. Why this exists — the problem and the three decisions

Publishing APIs to API Connect by hand — editing drafts in API Manager and pressing *Publish* — works for one team and a handful of APIs. It stops working when several teams share three separate API Connect instances with two catalogs each: nobody knows what is live where, changes cannot be reviewed before they hit a gateway, breaking changes slip through, and the instances drift apart.

GitOps turns this around. The Git repository *is* the catalog. Every API and every Product is a YAML file. A change is a pull request that is validated automatically. When it is merged, a pipeline logs in to the right instance with the `apic` toolkit and publishes exactly what is in Git.

The three design decisions

Question	Decision	Why
Should **Products** (plans, rate limits, visibility) be managed in Git too, or left to API Manager's auto-publish?	**Yes, always.** Every API sits next to a Product file that references it. The pipeline publishes Products, never bare APIs.	In API Connect v10 an API is only reachable on the gateway through a Product published to a Catalog, and the toolkit has no `apis:publish` — `products:publish` is the only publish command. Auto-publish creates opaque products you do not control.
Publish on **pull request** or on **merge**?	**PR = validate only; merge = publish.** PR builds never write to API Connect.	Two open PRs touching the same product would fight over the same `name:version` in a shared catalog. Validation is enough to give fast feedback; publishing belongs to the merged, reviewed state.
How to **promote** dev → integ → rc → prod when every environment is a *separate instance*?	**Four version folders per project** (`dev/`, `integ/`, `rc/`, `prod/`); each of the five catalogs is fed from one of them; promotion = a pull request that copies a folder to the next. Catalog-specific values stay *inside* the file.	Catalogs run different versions at different times, so the repository must be able to say "rc has 1.1.0, prod has 1.0.0". A folder per version makes that visible and reviewable, and every publish becomes a reviewed PR. Branch-per-environment would need cherry-picks; a single file for all catalogs could not express version skew. `-dev` and `-nightly` always show the same development state, so they share `dev/`.

Principles that follow from this

- Git is the single source of truth — including the ingress URLs of the three instances (`config/topology.yml`). API Manager becomes read-only for humans (viewer roles). Only the pipeline identities publish.
- Drafts in API Manager are not used at all — the PR *is* the draft.
- Files are byte-identical across folders; only the *version present* differs. Nobody hand-edits `integ/`, `rc/` or `prod/`.
- The pipeline never deletes, deprecates or retires anything. Removing an API from a gateway is a deliberate manual step.
- A breaking change is not forbidden — it just has to arrive as a new major version that coexists with the old one.

2. The big picture — three instances, four folders, five catalogs

Each environment is its own API Connect installation with its own ingress URLs, its own credentials and its own gateway. Inside every instance a project owns one or two catalogs, named after the project. Every catalog is fed from one of four **version folders** in the repository; `dev/` feeds two catalogs on two different instances.

dev instance

own cluster · own ingress · own API key

`projects/<project>/dev/`

`<project>-dev`
from dev/ · on merge of a PR that changed dev/

`projects/<project>/integ/`

`<project>-integ`
from integ/ · on merge of a promotion PR dev → integ

test instance

own cluster · own ingress · own API key

`projects/<project>/dev/`

`<project>-nightly`
from dev/ · same merge as -dev · no separate process

`projects/<project>/rc/`

`<project>-rc`
from rc/ · on merge of a promotion PR integ → rc

prod instance

own cluster · own ingress · own API key

`projects/<project>/prod/`

`<project>`
from prod/ · on merge of a promotion PR rc → prod

The path of a version

1 · Edit dev/
Developer changes `projects/orders/` on a branch, opens a PR.

→

2 · Validate
Four stages. Nothing is written to API Connect.

→

3 · Merge
→ `orders-dev` (dev instance) and `orders-nightly` (test instance).

→

4 · Promote
`promote.sh orders dev integ`; PR; merge → `orders-integ`.

→

5 · Promote
`promote.sh orders integ rc`; PR; merge → `orders-rc`.

→

6 · Promote
`promote.sh orders rc prod`; PR; merge → `orders`.

The four validation stages (fail fast, in this order)

1 YAML syntax
Can the file be parsed at all? Tabs, duplicate keys, bad indentation.
`yamllint`

2 OpenAPI validity
Is it a valid OpenAPI 3.0 document, and does it follow the APIC house rules?
`spectral`

3 APIC validity
Would API Connect accept the Product and its APIs (IBM extensions, assembly)?
`apic validate`

4 Compatibility
Does the new contract break existing consumers of *that folder's catalogs* without a major bump?
`oasdiff`

What is published, exactly?

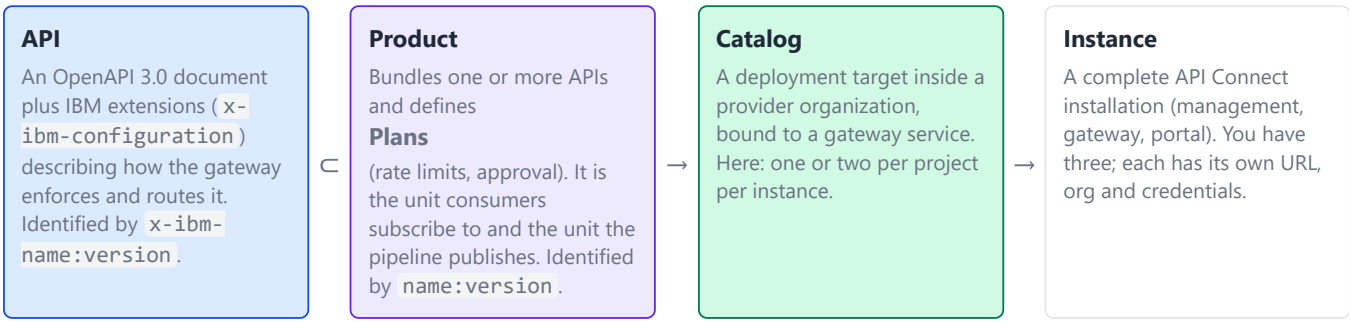
Only what changed, per folder. Each of the five catalogs remembers the last commit it received through a Git tag (`published/dev`, `published/nightly`, `published/integ`, `published/rc`, `published/prod`). The change set for a stage is "every product under `projects/*/<its folder>/` touched since its tag". A stage with nothing new is *Skipped*. The very first run (no tags yet) publishes every folder — the bootstrap.

Stage	Folder	Instance	Catalog	Published when
`dev`	`dev/`	dev	`<project>-dev`	a PR that changed `dev/` is merged into `main`
`nightly`	`dev/`	test	`<project>-nightly`	the same merge — `dev/` feeds both; nothing else to do
`integ`	`integ/`	dev	`<project>-integ`	a promotion PR `dev → integ` is merged
`rc`	`rc/`	test	`<project>-rc`	a promotion PR `integ → rc` is merged
`prod`	`prod/`	prod	`<project>`	a promotion PR `rc → prod` is merged

> **Where are the ingress URLs?**
> In `config/topology.yml`, once per instance: `ingress.management` (what the toolkit logs in to), `ingress.gateway` (what consumers call) and the provider `org`. Each stage points at an instance, so every folder is mapped to its ingress through that file. When an ingress changes, that is the only place to edit. Secrets never go there (section 11).

3. API Connect concepts you need

Five objects matter for this pipeline. If you are new to API Connect, this page is enough to follow the rest of the document.



Term	What it means for you
Instance / environment	dev, test or prod cluster. Declared in `config/topology.yml` with its ingress URLs and provider org; the API key comes from the variable group `apic-<instance>`.
Version folder	`projects/<project>/dev/`, `integ/`, `rc/` or `prod/` — a version of the project's definitions. `dev/` feeds `-dev` and `-nightly`; `integ/` feeds `-integ`; `rc/` feeds `-rc`; `prod/` feeds `<project>`.
Stage	One folder published to one catalog on one instance when a PR that changed the folder is merged. Five stages, declared in `config/topology.yml`.
Provider organization	The top-level container inside an instance. One per instance; its name is that instance's `APIC_ORG`.
Catalog	The target of a stage: `<project>-dev`, `-nightly`, `-integ`, `-rc` or `<project>`. Whatever is *published* in a catalog is live on its instance's gateway ingress.
Space	An optional sub-partition of a catalog with its own permissions. Not used here — projects are separated by having their own catalogs.
Plan	Part of a Product: rate limits and whether subscriptions need approval. Consumers subscribe to a plan.
Republish	Publishing a product `name:version` that already exists updates it in place; subscriptions are kept. Used for non-breaking changes.
Replace / supersede	Toolkit operations that move subscriptions from an old product version to a new one. Used manually after a breaking (major) change.

> **The one rule people forget**
>
> Changing `x-ibm-name` makes API Connect see a *new* API; the old one stays published until someone retires it. Treat `x-ibm-name` as permanent, and use `info.version` to evolve.

4. Repository layout

```
APIConnect-catalog/
├── .azure-pipelines/
│   ├── apic-ci.yml                    the pipeline: validate → 5 independent publish stages
│   └── templates/
│       ├── install-tools.yml          tooling; online (hosted agent) or offline (on-prem agent)
│       └── publish-stage.yml          one deployment stage per catalog
├── config/
│   ├── topology.yml                   instances (ingress URLs, org) and stages: folder → catalog → instance
│   ├── .yamllint.yml                  stage 1 rules
│   ├── spectral.yml                   stage 2 rules: OpenAPI 3.0 + APIC house rules
│   └── compat-ignore.txt              stage 4 reviewed exceptions
├── scripts/                           what the pipeline runs — also runnable on a laptop
├── tools/  install-offline.sh         installs the bundled binaries on an on-prem agent
├── docs/                              this guide (HTML source + PDF)
└── projects/
    └── <project>/                     one folder per team / product area = ownership + catalog-name prefix
        ├── project.yaml               optional: owner, catalog-name overrides
        ├── dev/                       development state → -dev (dev instance) and -nightly (test instance)
        │   ├── apis/<api>.yaml        OpenAPI 3.0.x + x-ibm-configuration
        │   └── products/<product>.yaml plans, rate limits, visibility; apis.*.$ref → ../apis/<api>.yaml
        ├── integ/                     → -integ (dev instance); written only by promote.sh dev integ
        ├── rc/                        → -rc (test instance); written only by promote.sh integ rc
        └── prod/                      → <project> (prod instance); written only by promote.sh rc prod
```

Why this shape

- **Project folder = catalog prefix.** `projects/orders/` publishes to `orders-dev`, `orders-nightly`, `orders-integ`, `orders-rc` and `orders`. If a catalog is named differently, declare it in `project.yaml`.
- **Version folder = what its catalog(s) have.** Looking at `projects/orders/prod/` answers "what does prod have?", `rc/` "what is the release candidate?", `integ/` "what is under integration test?", `dev/` "what is in development — and therefore in both `-dev` and `-nightly`?" — without opening API Manager.
- **`apis/` and `products/` side by side** so a Product references its APIs with an obvious relative `$ref` (`../apis/orders.yaml`) and the pipeline can map a changed API to the Product that must be republished.
- **No cross references.** A Product may only reference APIs in the same `<project>/<folder>/apis/`. Stage 3 rejects anything else, so a folder is always self-contained and publishable on its own.

Conventions enforced automatically

Rule	Enforced by
Files are valid YAML	Stage 1
OpenAPI 3.0.x; `info.x-ibm-name` is a lowercase slug; `info.version` is strict semver (`MAJOR.MINOR.PATCH`); exactly one `servers[]` entry; `x-ibm-configuration` present with an explicit `enforced`; gateway is `datapower-api-gateway`; no callbacks/links	Stage 2
Products reference APIs only via relative `$ref` into the same `<project>/<folder>/apis/`; `apic validate` passes for the product and every API	Stage 3
No breaking contract change without a MAJOR bump, judged per folder	Stage 4

Catalog values live in `x-ibm-configuration.properties` + `catalogs.<catalog>.properties` for all five catalogs; `integ/`, `rc/` and `prod/` are only ever written by `promote.sh`	Code review

5. Life of a change — from a dev edit to production

#	Who / what	What happens
1	Developer	Creates a branch, edits `projects/orders/dev/apis/orders.yaml` (say, adds an optional query parameter and bumps to 1.1.0), bumps the product version in `dev/products/orders.yaml`, opens a pull request to `main`.
2	Pipeline · *validate*	Computes the change set against `origin/main`: the changed API and the Product(s) in `dev/` that `$ref` it. Runs stages 1 → 4. Stage 4 compares the file with its version on `main`.
3	Pull request	The pipeline is a required status check. A red check blocks the merge; annotations point at file and line. The developer fixes and pushes; the pipeline reruns.
4	Reviewer	Reviews the diff like any code. Approves. Merge to `main`. **The merge is the publish decision.**
5	Pipeline · *validate* (main)	Same four stages, now per folder against the folder's last-published tag. Flags which of the five stages have something new: here `dev` and `nightly` (both fed from `dev/`); `integ`, `rc` and `prod` are skipped.
6	Pipeline · *publish_dev* and *publish_nightly* (in parallel)	Each stage reads its instance's ingress and org from `config/topology.yml`, logs in to **its** instance (dev, respectively test), recomputes the change set since its tag, runs the compatibility check against the **live** catalog (`orders-dev`, respectively `orders-nightly`), publishes, verifies `state: published`, moves its tag.
7	Developer	When development testing is done: `scripts/promote.sh orders dev integ`. The script copies `dev/` over `integ/` and prints the diff (1.0.0 → 1.1.0). Commit on a branch, open a **promotion PR**.
8	Pipeline · *validate* (PR)	Stage 4 now compares `integ/apis/orders.yaml` *as it was* (1.0.0, what `orders-integ` has) with the promoted 1.1.0 — the compatibility question is asked again for the integ consumers. Reviewer approves, merge → *publish_integ* publishes into `orders-integ` on the dev instance. Everything else is skipped.
9	Developer / Reviewer	`scripts/promote.sh orders integ rc`, PR, merge → *publish_rc* logs in to the **test instance** and publishes into `orders-rc`. `orders-nightly` is untouched — it only follows `dev/`.
10	Developer / Reviewer (/ Approver)	`scripts/promote.sh orders rc prod`, PR, merge → *publish_prod* logs in to the **prod instance** and publishes into `orders`. If you configured an approval on Environment `apic-prod`, the run waits for it first.

What if nothing under `projects/` changed?

The validate stage reports "no API/product changes detected", skips stages 1–4 and every publish stage is skipped. Changing only scripts or configuration therefore never touches API Connect — including a change of an ingress URL in `topology.yml`, which simply takes effect on the next publish.

What if two merges happen back-to-back?

The second run's change set includes everything since the tag, so it cannot "miss" the first merge. The *Exclusive lock* check on the Environments serialises publishes so two runs never publish into the same catalog concurrently.

6. The four validation stages

Stage	Tool & command	What it catches	How a failure looks
1 YAML	`yamllint -c config/.yamllint.yml`	Tabs, duplicate keys, broken indentation, unquoted special characters — anything that stops a parser before it even sees your API.	One annotation per finding with file, line and column.
2 OpenAPI	`spectral lint -r config/spectral.yml`	Invalid OpenAPI 3.0 structure (unknown fields, bad `$ref`, missing responses) plus the APIC house rules: `x-ibm-name`, semver, single server, `enforced`, gateway type, no callbacks/links.	Annotations per rule with the rule name (e.g. `apic-semver-version`) and a JUnit report in the *Tests* tab.
3 APIC	`apic validate <product.yaml>`	Repository policy first (relative `$ref`, file exists, same project *and* folder). Then the IBM toolkit validates the Product *and* every referenced API offline: IBM extensions, assembly policies, plan structure.	The toolkit's message, with the product file annotated.
4 Compat	`oasdiff breaking <baseline> <new>`	Contract changes that would break an existing consumer (section 7). Baseline per folder: the file at the folder's last publish (PR: on the target branch), or the version live in the target catalog (before publish).	A list of breaking changes (rule id, method, path) and one error annotation on the API file.

Example: what a developer sees

```
##[section]Stage 2 - spectral lint (1 file(s))
projects/orders/dev/apis/orders.yaml
  4:12  error  apic-semver-version  info.version must be strict semver (MAJOR.MINOR.PATCH)  info.version
  7:9   error  apic-single-server   The DataPower API Gateway supports a single servers[] entry  servers
##vso[task.logissue type=error;sourcepath=projects/orders/dev/apis/orders.yaml;linenumber=4]apic-semver-
version: …

[12:17:40] Environment 'prod': baseline 19cec99d…
##[section]Stage 4 - oasdiff breaking, mode=git (1 API(s))
1 changes: 1 error, 0 warning, 0 info
error [api-path-removed-without-deprecation] in API GET /orders/{id}
        api path removed without deprecation
##vso[task.logissue type=error]BREAKING change in orders-api (1.0.0 -> 1.1.0) without a MAJOR version bump.
  Bump MAJOR (new product version) or add a reviewed exception to config/compat-ignore.txt
```

> **Run the same checks on your laptop**
>
> Every stage is a script under `scripts/` with no Azure DevOps dependency. See the README section “Running the checks locally”. The `##vso` lines are harmless noise outside Azure DevOps.

7. Backward compatibility and versioning policy

Stage 4 is the gate that protects the consumers of your APIs — in *each* environment. It uses **oasdiff**, which understands OpenAPI 3.0 and ignores the `x-ibm-*` extensions, so changing an assembly policy or a backend URL is never mistaken for a contract change.

What counts as what

Class	Examples	Outcome
Breaking	Remove or rename a path, operation, request parameter or property · add a *required* request parameter or property · change a type or format · narrow an enum a client sends · remove a response property, status code or media type · tighten security · change the base path	Fails — unless `info.version` MAJOR was bumped
Potentially breaking	New response status code · new enum value in a response · marking an operation deprecated · loosening response constraints	Reported, does not fail
Non-breaking	New path or operation · *optional* request parameter or property · new response property · widening an enum a client sends · descriptions, examples, tags · anything under `x-ibm-configuration`	Passes

How the version number drives the pipeline

Non-breaking change
Keep the version or bump MINOR/PATCH. Same product version. The pipeline **republishes in place**; existing subscriptions keep working.

VS

Breaking change
Bump the API MAJOR (`1.4.2 → 2.0.0`) *and* the product version. The pipeline publishes a **new product version that coexists** with the old one. Consumers migrate on their schedule.

Stage 4 compares against the latest published version *with the same MAJOR*. A new MAJOR therefore has no baseline to break and is allowed through (the differences are still printed for the reviewer). Because the check runs per folder, a promotion PR asks the question again for the target catalog's consumers: 1.0.0 → 1.1.0 may be fine in dev and still be flagged in prod if prod skipped a step.

Exceptions

Sometimes a change that oasdiff calls breaking is acceptable — an endpoint nobody ever subscribed to, or a field documented as internal. Add a line to `config/compat-ignore.txt` **in the same pull request**, as specific as possible (method, path, fragment of the message), with the justification in the PR description. The exception is then reviewed together with the change, and is visible in history forever.

> **Do not**
> work around the gate by renaming `x-ibm-name`. That creates a second API and leaves the first one live and unmanaged.

8. Publishing — five independent stages, one folder each

Every publish stage answers one question: "does my folder have anything since my tag?" If yes it runs, if not it is *Skipped*. Stages never wait for each other; a merge that touches two folders publishes both in parallel.

Stage	Folder	Catalog · instance	Runs when
`publish_dev`	`projects/*/dev/`	`<project>-dev` · dev	a merge into `main` (or a manual run) and `dev/` changed since `published/dev`
`publish_nightly`	`projects/*/dev/`	`<project>-nightly` · test	same as `publish_dev`, against its own tag `published/nightly`. `dev/` feeds both catalogs — there is no separate nightly process.
`publish_integ`	`projects/*/integ/`	`<project>-integ` · dev	a merge (a promotion PR) when `integ/` changed since `published/integ`
`publish_rc`	`projects/*/rc/`	`<project>-rc` · test	a merge (a promotion PR) when `rc/` changed since `published/rc`
`publish_prod`	`projects/*/prod/`	`<project>` · prod	a merge (a promotion PR) when `prod/` changed since `published/prod`

The flag `changes.<stage>` is computed once by the validate stage for all five. PR builds set every flag to `false`, so nothing is ever published from a pull request.

Gates

The **pull-request review is the gate**: nothing reaches a catalog that was not merged into `main`. Every stage also runs as a deployment job on the Azure DevOps Environment `apic-<stage>`, so you can add a second gate where you want one — typically an *Approval* on `apic-prod` — without touching the pipeline. Put an *Exclusive lock* on all five Environments so two runs never publish into the same catalog at the same time.

Inside a publish stage

1. **Tools.** Online agents download them; offline agents must already have them (section 12).
2. **Instance settings.** `scripts/stage-env.sh --stage <stage>` reads `config/topology.yml`: stage → instance → `ingress.management` (the `apic --server` value) and `org`. A placeholder URL still containing `example.com` raises a warning.
3. **Instance credentials.** `credentials.json` of *this* instance from the Secure File `apic-toolkit-credentials-<instance>.json` → `apic client-creds:set`.
4. **Login.** `apic iam-apikey --server $APIC_SERVER --apiKey $APIC_APIKEY` with the key from the variable group `apic-<instance>`, followed by a read-only `catalogs:list` as a smoke test that also proves the org and the ingress are the right ones.
5. **Change set.** Products under `projects/*/<folder>/` touched since this stage's `published/<stage>` tag.
6. **Live compatibility check.** Stage 4 again, but the baseline is fetched from each product's target catalog with `apic apis:get`. This catches drift caused by anyone who published outside Git.
7. **Publish.** For each Product: catalog = pattern from `config/topology.yml` with the project name (or the `project.yaml` override); `apic products:publish --catalog <catalog> <product.yaml>` (with `--migrate_subscriptions` when the version already exists); then `products:get` must report `state: published`.
8. **Mark.** Move the `published/<stage>` tag to this commit and push it.
9. **Always.** Publish logs as a build artifact; `apic logout`.

Why tags instead of “publish what this run validated”?

Because stages fail independently. If the test instance is down while `dev/` receives three more merges, `publish_nightly` fails three times and then, once the instance is back, publishes the union of those changes in one go. Diffing against the stage's own tag gives exactly that, with no bookkeeping outside Git. A missing tag means “never published from Git” and selects every product of the folder — which is also how you bootstrap a new instance or a new catalog.

Rolling back

- **Non-breaking regression in integ, rc or prod:** the previous version is still in Git history — restore the folder in a PR (`git checkout <commit> -- projects/orders/prod`), or run the pipeline for an earlier commit (*Run pipeline → select commit*).
- **After a major version:** `apic products:replace` back to the previous product version on that instance, then retire the new one.

9. Promotion by pull request — dev → integ → rc → prod

A promotion is nothing more than "make the `integ/` folder look like the `dev/` folder". The helper does the copy so that nobody edits `integ/`, `rc/` or `prod/` by hand, and prints what is about to change and where it will go:

```
$ scripts/promote.sh orders dev integ
[06:57:07] Copied 2 file(s): projects/orders/dev -> projects/orders/integ
  product orders                        1.1.0
  api     orders-api                    1.1.0
 projects/orders/integ/apis/orders-api.yaml      | 16 ++++++++++++++-
 projects/orders/integ/products/orders.yaml      |  2 +-
Next: review 'git diff -- projects/orders/integ', commit on a branch, open a pull request.
Merging publishes: integ -> orders-integ on dev (https://platform-api.dev.apic.example.com)

$ git checkout -b promote/orders-1.1.0-integ
$ git commit -am "promote orders 1.1.0 dev -> integ"
$ git push -u origin HEAD          # then open the PR
```

What the PR shows and checks

- The diff between what the target catalog has today and what it will have — reviewable like any code change.
- Stage 4 with the **target folder's** baseline: the compatibility question for that catalog's consumers, independent of what happened upstream.
- Files that exist only in the target are kept and listed (use `--prune` to remove them). Remember that deleting a file never removes anything from API Connect.

Rules of the road

Rule	Why
Promote whole project folders, never single files	A Product and its APIs move together; a half-promoted folder fails stage 3 or publishes an inconsistent product.
Promote in order: dev → integ → rc → prod	The compatibility gate and the human review happen for each catalog's consumers. Skipping a step means the next catalog gets a change nobody tested at the previous level.
Never hand-edit `integ/`, `rc/` or `prod/`	Catalog differences belong inside the file (section 10). Hand edits create drift that the next promotion silently overwrites.
`-nightly` is not a promotion target	It is fed from `dev/` together with `-dev`. To change it, change `dev/`.
Hotfix in prod?	Fix in `dev/`, promote through integ and rc with quick PRs. If you truly must jump, promote `dev → prod` directly and back-fill integ and rc right after — the gate still runs against prod's baseline.
Several promotions in one PR are fine	`promote.sh orders integ rc` followed by `promote.sh orders rc prod` in the same branch publishes to both catalogs on merge; each stage checks its own folder against its own tag.

> **Can dev run ahead of prod for a long time?**
> Yes — that is the point of version folders. `dev/` can be at 2.0.0 while `prod/` stays at 1.4.2 for months. When 2.0.0 is promoted, prod receives a *new* product version next to the old one (section 7), and the old one is retired by hand later.

10. One file, five catalogs — catalog-specific properties

The backend URL, timeouts, or a header value differ between the five catalogs. Instead of editing the file per environment you declare a **property** with a default and override it per catalog *name*. The gateway substitutes `$(property)` at runtime with the value for the catalog it is running in — whichever instance that catalog lives on.

```
x-ibm-configuration:
  enforced: true
  gateway: datapower-api-gateway
  properties:
    target-url:
      value: https://orders-dev.internal.example.com      # default
      description: Backend base URL
  catalogs:                                                # overrides, keyed by the project's catalog names
    orders-dev:     { properties: { target-url: https://orders-dev.internal.example.com } }
    orders-nightly: { properties: { target-url: https://orders-nightly.internal.example.com } }
    orders-integ:   { properties: { target-url: https://orders-integ.internal.example.com } }
    orders-rc:      { properties: { target-url: https://orders-rc.internal.example.com } }
    orders:         { properties: { target-url: https://orders.internal.example.com } }
  assembly:
    execute:
      - invoke:
          version: 2.0.0
          target-url: $(target-url)$(request.path)          # resolved on the gateway
```

Because every catalog's value travels inside the file, one `dev/` file serves `-dev` and `-nightly` at once, `promote.sh` is a pure copy, the folders stay byte-identical apart from the version being promoted, and the compatibility gate never sees an environment difference as a contract change (everything under `x-ibm-configuration` is ignored by oasdiff).

> **Verify once per instance**
>
> Catalog-specific property overrides are an API Connect feature of the API definition. Publish the sample (it lands in `-dev` and `-nightly` at once) and confirm the invoke target differs between the two before relying on it for prod. Alternative if needed: define the property at catalog level in API Manager (Catalog settings → Properties) on each instance and reference it the same way.

What must never differ between catalogs

- Paths, parameters, schemas — the contract. If test and prod need different contracts they are different major versions.
- Security schemes. A prod-only header requirement would be invisible to the compatibility gate in the lower catalogs.
- Product plans and rate limits — they are part of the reviewed artifact. If prod needs stricter limits, that is a product change reviewed in a PR and promoted like everything else.

11. Security — ingress URLs in Git, secrets in Azure DevOps

Item	Where it lives	Notes
IBM Cloud API key (`APIC_APIKEY`)	Azure Key Vault, surfaced through the variable groups `apic-dev`, `apic-test`, `apic-prod`	One key per *instance*, owned by a service ID that is a member of that instance's provider org with a publish role. Masked in logs; only passed via `env:`. The prod key never exists on the dev stages.
Ingress URLs, org name (`ingress.management`, `ingress.gateway`, `org`)	`config/topology.yml`, per instance — versioned in Git	Not secret. Kept in the repository so a changed ingress is a reviewed one-line PR and every folder is visibly mapped to its instance. A fork can only *read* the URLs; without the API key and the toolkit credentials of that instance it cannot publish.
Toolkit credentials (`credentials.json`)	Secure Files `apic-toolkit-credentials-dev.json`, `-test.json`, `-prod.json`	Downloaded from each instance's API Manager → *Tools for download*. Contains the toolkit client id/secret of that instance; never committed.
Session token	`~/.apiconnect` on the agent during the job	`apic logout` runs in an `always()` step. On self-hosted agents, run the agent as a dedicated user.
Repository write access (tags)	Build identity	Needs *Contribute* + *Create tag*; on GitHub a token with `contents:write`. Nothing else is written to Git by the pipeline.
Humans in API Manager	Provider org roles, per instance	Viewer on catalogs after go-live. If nobody can click *Publish*, the live-compatibility check never finds drift.

> **Never**
> run `apic --debug` in the pipeline (it can dump tokens), never echo `$APIC_APIKEY`, and never commit `credentials.json`, `.apiconnect/` or `.netrc` — the `.gitignore` blocks the obvious names, not intent.

Rotation

IBM Cloud API keys do not expire unless you set a lifetime. Decide a rotation cadence (e.g. yearly), rotate in Key Vault, and let the next run pick it up — nothing in the repository changes. If an instance instead requires a toolkit API key created with `apic api-keys:create`, it carries a `ttl`: put the expiry in a calendar, because an expired key fails the login step with an unhelpful message.

12. Setting up an on-prem agent from the offline bundle

The bundle ships everything the pipeline needs on a **Linux x64** agent that has no internet access, except the IBM toolkit (licensed, downloaded from your own instances). One agent serves all three instances — it only needs network access to their management endpoints.

```
APIConnect-catalog-bundle-<date>/
├── repo/                          the repository, ready to push to your on-prem Git server
├── APIConnect-catalog.gitbundle    the same repository with history:  git clone APIConnect-catalog.gitbundle
├── tools/
│   ├── install-offline.sh          installer (also in repo/tools/)
│   └── linux-x64/
│       ├── bin/  yq · oasdiff · jq · spectral      static binaries; spectral is standalone (no Node.js)
│       ├── wheels/  yamllint · pyyaml · pathspec    for CPython 3.9 – 3.14, pip --no-index
│       ├── SHA256SUMS                verified by the installer
│       └── MANIFEST.txt              versions and sources
└── docs/  guide.pdf · guide.html
```

Steps

1. **Prepare the machine.** Linux x64 with bash 5, git 2.x, GNU coreutils, Python 3.9+ with pip, and the Azure DevOps agent installed as a dedicated user. Outbound access to the three management endpoints.
2. **Install the bundled tools.**

   ```
   unzip APIConnect-catalog-bundle-*.zip && cd APIConnect-catalog-bundle-*
   sudo bash tools/install-offline.sh             # verifies SHA256SUMS, installs to /usr/local/bin, pip installs yamllint
   ```

3. **Install the IBM toolkit.** In API Manager (any instance; use the highest fix pack if they differ) click *Tools for download*, download the Linux CLI.

   ```
   sudo install -m 0755 apic-slim /usr/local/bin/apic    # or `apic` if you downloaded CLI + Designer
   apic --accept-license --live-help=false version       # must match the instances, 10.0.8.x
   ```

   Both flags answer the toolkit's two first-run questions (license, live help) and persist them for the user running the command — run it once as the agent user (Dockerfile: same `USER`), otherwise every call blocks on stdin in a pipeline step. The scripts pass both flags on every call regardless (`scripts/lib.sh`); see the runbook, step 2.

4. **Verify as the agent user.** `for t in yamllint spectral yq oasdiff jq apic git realpath; do command -v $t; done` — every line must print a path.
5. **Push the repository** to your on-prem Git (Azure Repos or GitHub Enterprise): `git clone APIConnect-catalog.gitbundle APIConnect-catalog`, add the remote, push `main`.
6. **Tell the pipeline it is offline.** In variable group `apic-shared` set `TOOLS_MODE = offline`. The tooling step then only checks that the tools exist and never reaches the internet; a missing tool fails the job with a clear message.
7. **Point the pipeline at your pool.** In `apic-ci.yml` set `AGENT_POOL` to your pool name and keep `APIC_TOOLKIT_SOURCE: preinstalled`. Nothing else in the pipeline changes.
8. **Azure DevOps Server specifics** (branch policy instead of the `pr:` trigger, secret variables instead of Key Vault, build-service permissions, certificates) are covered step by step in `docs/onprem-setup.pdf`.

> **Updating tools later**
> Rebuild the bundle on a connected machine (same layout, regenerate `SHA256SUMS`), copy it over, rerun the installer. Versions are listed in `MANIFEST.txt`; the pipeline prints the versions it found at the start of every job.

13. Setting up Azure DevOps

#	Object	Configuration
1	**Pipeline**	New pipeline from existing YAML: `.azure-pipelines/apic-ci.yml`. For Azure Repos add a branch policy on `main`: *Build validation* → this pipeline, required. For GitHub the `pr:` trigger applies; make the check required in branch protection.
2	`config/topology.yml`	Replace every `CHANGE ME`: for each instance the `ingress.management` URL (what `apic --server` takes — the platform-API / API Manager ingress), the `ingress.gateway` URL and the provider `org`. This is the only place ingress URLs live; the pipeline warns if a placeholder is still present.
3	**Variable group** `apic-shared`	`APIC_TOOLKIT_VERSION` = e.g. `10.0.8.9` · `TOOLS_MODE` = `offline` (on-prem) or `online`.
3b	**Variable groups** `apic-dev`, `apic-test`, `apic-prod`	One per instance, Key Vault-linked, holding only the secret `APIC_APIKEY`. Grant the pipeline access when prompted on first run.
4	**Secure files** `apic-toolkit-credentials-dev.json`, `-test.json`, `-prod.json`	Pipelines → Library → Secure files. Upload each instance's `credentials.json`. Authorize the pipeline.
5	**Environments** `apic-dev`, `apic-nightly`, `apic-integ`, `apic-rc`, `apic-prod`	Pipelines → Environments (created on first run). Add *Exclusive lock* on all five. Approvals are optional — the PR review is the gate; add one on `apic-prod` if you want a second pair of eyes before production.
6	**Agent pool**	Self-hosted pool with the prepared Linux agent (section 12), or Microsoft-hosted `ubuntu-latest` with `TOOLS_MODE=online` and the toolkit as a Universal Package in an Artifacts feed.
7	**Repository permissions**	The build identity (*Project Collection Build Service* or the project's build service) needs *Contribute* and *Create tag* on the repository so `published/<stage>` tags can be pushed.
8	**Catalogs** (API Connect side)	On each instance, create the catalogs the pattern expects: `<project>-dev` and `-integ` on dev, `-nightly` and `-rc` on test, `<project>` on prod, each bound to the gateway service behind that instance's `ingress.gateway`. Different names → `projects/<project>/project.yaml`.

First run

1. Open a pull request that touches only the sample project (e.g. edit a description in `dev/`). Confirm the four stages run and the check turns green.
2. Merge. Watch `publish_dev` and `publish_nightly`: each should report “No tag published/… yet: bootstrapping”, publish every product of `dev/` to `example-project-dev` (dev instance) and `example-project-nightly` (test instance), and create its tag. `publish_integ`, `publish_rc` and `publish_prod` bootstrap their folders in the same run (waiting for an approval only where you configured one).
3. In each API Manager, open the catalogs and confirm the products are *Published*. Call the API through each instance's gateway ingress.
4. Delete `projects/example-project` in a PR once your first real API is in. Remember: deleting the files does not remove the products from the five catalogs — retire the sample product by hand.

14. Day-2 operations

Add a new API

1. Create `projects/<project>/dev/apis/<api>.yaml` — OpenAPI 3.0.x with `info.x-ibm-name`, semver `info.version`, a single `servers[].url` and an `x-ibm-configuration` block with `catalogs.*` overrides for all five catalog names. Copy the sample as a starting point.
2. Either add it to an existing Product in `dev/products/` (`apis: { <api>: { $ref: ../apis/<api>.yaml } }`) or create a new Product there.
3. Open a PR. Stage 4 reports "NEW … no baseline" and passes. Merge → `-dev` and `-nightly`.
4. Promote `dev → integ → rc → prod` when ready (section 9).

Add a new project

Create the five catalogs on the three instances (or declare different names in `project.yaml`), then add `projects/<project>/dev/{apis,products}/`. `integ/`, `rc/` and `prod/` appear with the first promotions.

Make a non-breaking change

Edit in `dev/`, optionally bump MINOR/PATCH, open a PR. On merge the Product is republished in place in `-dev` and `-nightly`; promote to `integ/`, `rc/`, `prod/` when ready.

Make a breaking change

1. In `dev/`, bump the API `info.version` MAJOR (e.g. `2.0.0`) and the Product `info.version`. Consider a new product file if the old version must stay available for a long time.
2. PR → stage 4 prints the breaking changes but passes because MAJOR changed. Merge, approve.
3. Promote through integ, rc and prod. In each catalog both versions are now live. Communicate the deprecation; when consumers have moved, migrate remaining subscriptions (`apic products:replace` / `supersede`) and retire the old version manually, instance by instance.

Change plans or rate limits

Edit the Product file in `dev/` only, then promote folder by folder. It is republished in place; the gateway applies the new limits.

Add a compatibility exception

Add one line to `config/compat-ignore.txt` in the same PR as the change, matching the oasdiff message as specifically as possible (method + path + text fragment). Explain why in the PR.

Retire an API or Product (manual, by design)

1. Remove the files from the folders in PRs (usually `dev/` first, `prod/` last). The pipeline warns "Deleted in Git but NOT removed from APIC".
2. On each instance, in API Manager or with the toolkit: deprecate → retire → delete the product version, per catalog.

Roll back

Restore the previous file in the folder via a PR (it is in Git history: `git checkout <commit> -- projects/orders/prod`), or *Run pipeline → select an earlier commit*. For a major version, `products:replace` back and retire the newer version.

Change an ingress URL

Edit the instance's `ingress.management` / `ingress.gateway` in `config/topology.yml`, open a PR, merge. No publish happens (nothing under `projects/` changed); the next publish to that instance uses the new URL. Run `eval "$(scripts/stage-env.sh --stage <stage>)" && apic catalogs:list --server "$APIC_SERVER" --org "$APIC_ORG"` from the agent to check reachability first.

Upgrade an instance / the toolkit

After an instance upgrade, download the matching CLI, install it on the agent (offline) or publish it to the Artifacts feed (online) and bump `APIC_TOOLKIT_VERSION`. If instances are temporarily on different fix packs, use the toolkit of the highest one and check the versions step in the first run against each instance.

15. Troubleshooting — reading a failed run

Symptom	Meaning and fix
"No API/product changes detected" and everything skipped	Only files outside `projects/` changed, or files outside `<project>/<folder>/{apis,products}/` (they are ignored with a log line). Expected.
A publish stage shows *Skipped*	Its folder has nothing new since `published/<stage>`. Expected — e.g. a promotion PR into `rc/` skips `dev`, `nightly`, `integ` and `prod`.
`-nightly` differs from `-dev`	Both are fed from `dev/`, so they only differ when one of the two stages failed (e.g. the test instance was unreachable). Fix the cause and re-run the failed stage; it publishes everything since its own tag.
Stage-env warning "still uses the placeholder ingress … example.com"	`config/topology.yml` still has a `CHANGE ME` URL for that instance. Replace it and merge.
Stage 1 `found character '\t' that cannot start any token`	A tab in the YAML. Replace with spaces.
Stage 2 `apic-x-ibm-name` / `apic-semver-version`	Add `info.x-ibm-name` (lowercase slug) or make `info.version` `MAJOR.MINOR.PATCH`.
Stage 2 `apic-single-server`	Only one `servers[]` entry is allowed; its path is the base path.
Stage 3 "must use a relative $ref …"	The Product references an API by `name:version`. Use `$ref: ../apis/<file>.yaml`.
Stage 3 "points outside projects/…/integ/apis/ (cross-project / cross-folder …)"	A Product in one folder references an API in another. Each folder must be self-contained; run `promote.sh` instead of hand-linking.
Stage 3 `apic validate` error text	IBM extension or assembly problem; the message is the toolkit's. Run `apic validate <product>` locally to iterate quickly.
Stage 4 "BREAKING change … without a MAJOR version bump" on a *promotion* PR	The target catalog's consumers would break: the version now in `integ/`, `rc/` or `prod/` is not compatible with the one that was there. Promote a MAJOR version instead, or add a reviewed exception (section 7).
Stage 4 "oasdiff failed (exit 2)"	oasdiff could not load one of the specs — usually a `$ref` to a missing component. Fix the spec; stage 2 normally catches this first.
Publish: "offline mode: 'X' is not installed on this agent"	Run `tools/install-offline.sh` on the agent, or install the toolkit; check PATH for the agent user.
Publish: login fails	Check `ingress.management` of *that* instance in `config/topology.yml` (the pipeline prints the resolved URL in the "Resolve instance ingress" step), that the API key's service ID is a member of that instance's provider org, and that the key is not expired. Try `eval "$(scripts/stage-env.sh --stage <stage>)"` plus the login command interactively from the agent.
Publish: `catalogs:list` works but publish says catalog not found	The catalog name derived from the project folder (`<project>-integ` etc.) does not exist on that instance. Create it, or declare the real name in `projects/<project>/project.yaml`.
Publish: "Refusing to publish: … is not under projects/<project>/prod/products/"	Safety guard: a stage only publishes its own folder (and `nightly` only `dev/`). Should not happen from the pipeline; happens when running `publish.sh` by hand with the wrong list.

Publish: live compatibility check fails but the PR passed	What is live in the catalog differs from what Git thinks — someone published outside Git on that instance. Reconcile: export the live definition, compare, decide, then republish from Git.
Publish: "ended in state 'staged'/'pending'"	The product uploaded but did not activate. Check the catalog's gateway service and any approval/task settings in Catalog settings on that instance.
Publish: `git push … refs/tags/published/…` permission denied	Grant the build identity *Contribute* + *Create tag*. The publish itself already happened; rerun the stage to set the tag.
Publish to `<project>` (prod) rejected for an existing version	The prod catalog may be in *Production mode*, which can block republishing the same version. Either disable it or adopt "every prod change bumps the version + `products:replace`".

16. Before the first publish — items to verify on each instance

The pipeline uses commands documented for the v10.0.8 toolkit. A few behaviours depend on your installations and were flagged in the design; check them once **per instance** from a laptop with the toolkit installed.

	Check	How
☐	Ingress URLs and org in `config/topology.yml`	No `CHANGE ME` left. `eval "$(scripts/stage-env.sh --stage dev)"` prints the dev instance's values; repeat for `nightly` (test instance) and `prod`.
☐	Non-interactive login works on dev, test and prod	`apic iam-apikey --server <ingress.management> --apiKey <IBM Cloud API key>` then `apic catalogs:list --server … --org …`. If an instance only accepts IBMid/OIDC, switch the login step to `apic login --server … --sso --context provider --apiKey <toolkit key from apic api-keys:create>`.
☐	Catalog names	`apic catalogs:list` on each instance shows `<project>-dev` / `-integ`, `-nightly` / `-rc`, `<project>` — or you have written the real names into `project.yaml`.
☐	Shape of `apis:get` output	`apic apis:get --scope catalog <name>:<version> --catalog <project>-dev --org … --server … --format yaml --output -`. The script accepts the bare definition or an envelope with `.api`; look at the first live-compat log.
☐	Shape of `apis:list --format json`	Expected `{ "results": [ … ] }`; the script also accepts a bare array.
☐	`apic validate` exit code	Break a sample on purpose; the command must exit non-zero. The script additionally counts “Validated” lines.
☐	`--migrate_subscriptions` on republish	Republish an existing version in `<project>-dev` with a test subscription and confirm it survives.
☐	Catalog-specific properties resolve	Merge the sample; it lands in `-dev` and `-nightly`. The invoke target must differ between the two gateways.
☐	Production mode on the prod catalogs	Catalog settings on the prod instance. Decide the policy (section 15, last row).
☐	Gateway OpenAPI 3.0 subset	Optionally run `apic draft-apis:validate` online in `<project>-dev` for one representative API.
☐	Toolkit version vs. instance versions	`apic version` against each management endpoint; if fix packs differ, use the highest toolkit and test publish on the lowest instance.
☐	Existing content in the catalogs	Anything published today that is not in Git will be flagged as drift by the live check, or silently replaced if a product with the same `name:version` is in Git. Export or retire it first (your manual “start clean”).

17. Appendix — scripts, tools and versions

Configuration

File	Role
`config/topology.yml`	Instances with `ingress.management` (= `apic --server`), `ingress.gateway` and `org`; and the five stages: folder → catalog pattern (`{project}-dev` …) → instance. Read by every script; mirrored by the stage list in `apic-ci.yml`.
`projects/<project>/project.yaml`	Optional: owner, and `catalogs: { <stage>: <name> }` overrides when a catalog does not follow the pattern.

Scripts

Script	Role
`scripts/lib.sh`	Shared helpers: logging, Azure DevOps annotations, path → project/folder, stage → folder/instance/catalog, instance → ingress/org.
`scripts/stage-env.sh --stage S [--vso]`	Prints the instance settings behind a stage (`APIC_SERVER`, `APIC_ORG`, `APIC_GATEWAY`) from `topology.yml`; used by the pipeline before login and by humans for local checks.
`scripts/changeset.sh --mode pr|main|stage …`	Stage 0 orchestrator: per-folder change sets and baselines; in main mode also the per-stage "has changes" flags.
`scripts/changed-products.sh BASE HEAD OUT --folder F`	Git diff → Product list for one folder; deleted files → warning only.
`scripts/product-apis.sh --list FILE`	Product list → referenced API files.
`scripts/validate-yaml.sh LIST…`	Stage 1, yamllint.
`scripts/validate-oas.sh APIS`	Stage 2, spectral; writes `out/spectral.xml`.
`scripts/validate-apic.sh PRODUCTS`	Stage 3, `$ref` policy (same project and folder) + `apic validate`.
`scripts/validate-compat.sh OUT_DIR`	Stage 4 in git mode, once per folder against its own baseline.
`scripts/compat-check.sh --mode git|live …`	oasdiff for one list; live mode fetches baselines from each product's catalog for the given stage.
`scripts/publish.sh --stage S PRODUCTS`	`apic products:publish` into `catalog_for(project, stage)` + state assertion; refuses files outside the stage's folder.
`scripts/promote.sh PROJECT FROM TO [--prune]`	Copies a version folder to the next one (`dev → integ → rc → prod`), shows the diff and the catalog/instance the merge will publish to — the promotion PR helper.
`tools/install-offline.sh`	Installs the bundled binaries and yamllint on an on-prem agent.

Bundled tool versions (Linux x64)

Tool	Version	Purpose
yq	v4.53.6	YAML queries in scripts (mikefarah/yq)

oasdiff	v1.32.1	Stage 4 backward-compatibility
jq	1.8.2	JSON parsing of toolkit output
spectral	v6.16.3 (standalone)	Stage 2 OpenAPI lint; no Node.js needed
yamllint	1.38.0 (1.37.1 for Python 3.9)	Stage 1 YAML syntax; wheels for CPython 3.9–3.14
apic toolkit	10.0.8.x — **not bundled**	Download from your instances (API Manager → Tools for download)

Toolkit commands used (v10.0.8 CLI reference)

`apic client-creds:set` · `apic iam-apikey` (or `apic login --sso --apiKey`) · `apic catalogs:list` · `apic validate` · `apic apis:list` · `apic apis:get` · `apic products:publish` · `apic products:get` · `apic logout`. Manual lifecycle: `apic products:replace` · `products:supersede` · `products:delete`.
